字符串

字符串的常用内置函数

- 注意：与 len(s)、max(s)、min(s) 等把要处理的字符串放在参数中的函数不同，下面所有的函数都是字符串的内置函数，所以在使用时，要写成`string.function(args)`的形式，如`s.lower()`，而不是`lower(s)`。
- 注意：下面所有的函数，并不会修改字符串本身。如果函数的作用是修改字符串，也是通过返回值返回一个新字符串，例如`r = s.lower()`，是把 s 中的大写字母转成小写字母，新字符串保存在 r 中，而 s 不会发生任何变化。

字符串状态获取函数

方法名	关键词	功能	参数	返回值
islower()	是否全小写	判断字符串是否是全小写字母组成	无	结果是True或False
isupper()	是否全大写	判断字符串是否是全大写字母组成	无	结果是True或False
isdigit()	是否全数字	判断字符串是否是由纯数字组成	无	结果是True或False
isalpha()	是否全字母	判断字符串是否是由纯字母组成	无	结果是True或False
isalnum()	是否全数字字母	判断字符串是否是由纯数字和字母组成	无	结果是True或False
istilte()	是否单词首字母大写	判断字符串是否是满足单词首字母大写格式	无	结果是True或False
startswith(str)	判定前缀	判断字符串是否以指定字符串开始	str:用于作为判定条件的字符串对象	结果是True或False
endswith(str)	判定后缀	判断字符串是否以指定字符串结束	str:用于作为判定条件的字符串对象	结果是True或False

```
str1 = "itcast"
str1.islower()  # True
str2 = "ITHEIMA"
str2.islower()  # False

str1 = "itcast"
str1.isupper()  # False
str2 = "ITHEIMA"
str2.isupper()  # True

str1 = "666"
str1.isdigit()  # True
```

```
str2 = "itcast666"
str2.isdigit()  # False

str1 = "itcast"
str1.isalpha()  # True
str2 = "itcast666"
str2.isalpha()  # False

str1 = "itcast666"
str1.isalnum()  # True
str2 = "itcast_666"
str2.isalnum()  # False

str1 = "It Cast"
str1.istitle()  # True
str2 = "ItCast"
str2.istitle()  # False

str1 = "logo.png"
str1.startswith("lo")  # True

str2 = "logo.png"
str2.endswith(".png")  # True
```

字符串转换函数

方法名	关键词	功能	参数	返回值
lower()	转小写	字符串中所有字母转小写字母，支持英文字母	无	结果是一个字符串
casefold()	转小写	字符串中所有字母转小写字母，支持各种语言	无	结果是一个字符串
upper()	转大写	字符串中所有字母转大写字母	无	结果是一个字符串
swapcase()	大小写互换	字符串中字母大写转小写，小写转大写	无	结果是一个字符串
title()	单词首字母大写其他小写	字符串中每个单词首字母大写，其余字母小写（区分单词以空格区分）	无	结果是一个字符串
capitalize()	字符串首字母大写其他小写	字符串首个字母大写，其余字母小写	无	结果是一个字符串

```
# 注意下面所有的str1和str2，在执行函数后，结果放在返回的新字符串中，原字符串不会发生变化！
str1 = "itcast"
str1.lower()       # "itcast"
str2 = "ITCAST666"
str2.lower()       # "itcast666"
```

```
str2                # "ITCAST666"

str1 = "itcast"
str1.upper()        # "ITCAST"
str2 = "ITCAST666"
str2.upper()        # "ITCAST666"

str1 = "IT heima"
str1.swapcase()     # "is HEIMA"
str2 = "Itcast666"
str2.swapcase()     # "iTCAST666"

str1 = "itcast"
str1.title()        # "Itcast"
str2 = "it hei ma"
str2.title()        # "It Hei Ma"

str1 = "itcast"
str1.capitalize()   # "Itcast"
str2 = "it hei ma"
str2.capitalize()   # "It hei ma"
```

字符串格式转换函数

方法名	关键词		参数	返回值
strip (str)	左右去字符	去掉字符串左右两侧在参数字符串中包含的所有字符	str:包含了若干个字符的字符串	结果是一个字符串
lstrip (str)	左侧去字符	去掉字符串左侧在参数字符串中包含的所有字符	str:包含了若干个字符的字符串	结果是一个字符串
rstrip (str)	右侧去字符	去掉字符串右侧在参数字符串中包含的所有字符	str:包含了若干个字符的字符串	结果是一个字符串
ljust (len,str)	右侧补占位符	使用指定字符在原始字符串右侧补充到长度为指定值	len:补充字符后的字符串长度str:补充的字符，如果使用多个字符组成的字符串将报错	结果是一个字符串
rjust (len,str)	左侧补占位符	使用指定字符在原始字符串左侧补充到长度为指定值	len:补充字符后的字符串长度str:补充的字符，如果使用多个字符组成的字符串将报错	结果是一个字符串
center (len,str)	左右两侧补占位符	使用指定字符在原始字符串两侧补充到长度为指定值，左侧补充数量≥右侧补充数量	len:补充字符后的字符串长度str:补充的字符，如果使用多个字符组成的字符串将报错	结果是一个字符串

方法名	关键词		参数	返回值
zfill(len)	左侧补0	使用0在原始字符串左侧补充到长度为指定值，小数点占1位	len:补充字符后的字符串长度	结果是一个字符串

```
# 注意下面所有的str1,str2和str3，在执行函数后，结果放在返回的新字符串中，原字符串不会发生变化！
str1 = "  itcast  "
str1.strip(" ")         # "itcast"
str2 = "itcast"
str2.strip("it")        # "cas"
str3 = "itcast"
str3.strip("sat")       # "itc"

str1 = "  itcast  "
str1.rstrip(" ")         # "  itcast"
str2 = "itcast"
str2.rstrip("it")        # "itcas"
str3 = "itcast"
str3.rstrip("sat")       # "itc"

str1 = "itcast"
str1.ljust(10,"6")      # "itcast6666"
str1.ljust(5,"6")       # "itcast"
str1.ljust(10,"66")     # 报错

str1 = "itcast"
str1.rjust(10,"6")      # "6666itcast"
str1.rjust(5,"6")       # "itcast"
str1.rjust(10,"66")     # 报错

str1 = "itcast"
str1.center(10,"6")      # "66itcast66"
str1.center(11,"6")      # "666itcast66"
str1.center(5,"6")       # "itcast"
str1.center(10,"66")     # 报错

str1 = "3.14"
str1.zfill(6)            # "003.14"
str2 = "314"
str2.zfill(6)            # "000314"
str3 = "3.14"
str3.zfill(3)            # "3.14"
```

字符串拆分函数

方法名	关键词	功能	参数	返回值
partition(str)	左侧查询3段拆分	从字符串左侧查找到参数后，将参数左侧、参数、参数右侧的三个字符串组成元组并返回	str:比对字符串	包含三个字符串的元组对象

方法名	关键词	功能	参数	返回值
rpartition(str)	右侧查询3段拆分	从字符串右侧查找到参数后，将参数左侧、参数、参数右侧的三个字符串组成元组并返回	str:比对字符串	包含三个字符串的元组对象
split(str)	多段拆分	使用参数作为分割线将原始字符串拆分成若干个字符串并组织成列表返回	str:比对字符串	包含多个字符串的列表对象
splitlines ()		使用换行符作为分割线将原始字符串拆分成若干个字符串并组织成列表返回	无	包含多个字符串的列表对象

```
str1 = "hello python"
str1.partition("o")     # ('hell', 'o', ' python')
str1.rpartition("o")    # ('hello pyth', 'o', 'n')
str1.partition("a")     # ('hello python', '', '')
str1.rpartition("a")    #  ("", "", "hello python")

str1 = "hello python"
str1.split("o")         # ["hell", " pyth", "n"]
str1.split("oo")        # ["hello python"]

str1 = "hello\nitcast\npython"
str1.splitlines()     # ["hello", "itcast", "python"]
str2 = "hello itcast"
str2.splitlines()     # ["hello itcast"]
```

字符串连接函数

方法名	关键词	功能	参数	返回值
join(str)	占位连接	将原始字符串填充到参数的每个字符之间组成新的字符串返回	str:待填充的字符串	结果是一个字符串
str1 + str2	字符串连接	将两个字符串按照顺序拼接成一个新的字符串返回	无	结果是一个字符串

```
"6".join("itcast")          # "i6t6c6a6s6t"
"66".join("itcast")         # "i66t66c66a66s66t"
```

字符串查询函数

方法名	关键词	功能	参数	返回值
find(str,begin,end)	左侧查找	从左侧查找字符串从指定开始位置到指定结束位置间第一次出现的索引位置	str:要查找的字符串，begin:开始索引，整数,该值要小于end，否则结果为-1	结果是一个int整数，如没有查找到返回-1

方法名	关键词	功能	参数	返回值
rfind(str,begin,end)	右侧查找	从右侧查找字符串从指定开始位置到指定结束位置间第一次出现的索引位置	str:要查找的字符串，begin:开始索引，整数,该值要小于end，否则结果为-1	结果是一个int整数，如没有查找到返回-1
index(str,begin,end)	左侧查找	从左侧查找字符串从指定开始位置到指定结束位置间第一次出现的索引位置	str:要查找的字符串，begin:开始索引，整数,该值要小于end，否则结果为-1	结果是一个int整数，如没有查找到返回-1
rindex(str,begin,end)	右侧查找	从右侧查找字符串从指定开始位置到指定结束位置间第一次出现的索引位置	str:要查找的字符串，begin:开始索引，整数,该值要小于end，否则结果为-1	结果是一个int整数，如没有查找到返回-1
count(str)	字符串数量	查询指定字符串在原始字符串中出现的次数	str:被查询的字符串	结果是一个int整数

```
str1 = "itcast"
str1.find("t")        # 1
str1.find("t",3,6)    # 5
str1.find("t",6,3)    # -1
str1.find("cast")     # 2
str1.find("m")        # -1

str1 = "itcast"
str1.rfind("t")        # 5
str1.rfind("t",0,4)    # 1
str1.rfind("t",4,0)    # -1
str1.rfind("cast")     # 2
str1.rfind("m")        # -1

str1 = "itcast"
str1.index("t")        # 1
str1.index("t",3,6)    # 5
str1.index("t",6,3)    # 报错
str1.index("cast")     # 2
str1.index("m")        # 报错

str1 = "itcast"
str1.rindex("t")        # 5
str1.rindex("t",0,4)    # 1
str1.rindex("t",4,0)    # 报错
str1.rindex("cast")     # 2
str1.rindex("m")        # 报错

str1 = "itcast"
```

```
str1.count("t")        # 2
str1.count("tc")       # 1
```

字符串替换函数

方法名	关键词	功能	参数	返回值
replace(old_str,new_str,num)	替换	使用新字符串替换原始字符串中的指定字符串信息	old_str:被替换的字符串 new_str:新字符串num:替换数量	结果是一个字符串
expandtabs()	制表位\t换空格	使用空格替换原始字符串中的制表位\t	无	结果是一个字符串

```
str1 = "itcast"
str1.replace("t","T")     # "iTcasT"
str1.replace("t","T",1)   # "iTcast"
str1.replace("T","t")     # "itcast"

str1 = "66"
str2 = "itca\tst"
str1+str2                   # "66itca  st"  （空白是一个\t）
str1+str2.expandtabs()      # "66itca    st"（空白是4个空格）
```

字符串其它函数

方法名	关键词	功能	参数	返回值
encode(charset)	编码	以指定的字符集将字符串进行编码	charset: 字符集	编码后的二进制编码（显示为十六进制形式）
decode(charset)	解码	以指定的字符集将字符串进行解码	charset: 字符集	解码后的字符串
maketrans(str1,str2)	生成字典	使用两个等长的字符串中的每个对应位置的字符生成一个字典对象	str1:用于生成字典中key的字符对应的字符串，长度必须与str2相同。 str2:用于生成字典中value的字符对应的字符串，长度必须与str1相同	包含字符串对应位置字符组成的字典对象
translate (dict)	使用字典关系转换	使用字典对字符串信息进行转换	dict:包含有转换关系的字典对象	结果是一个字符串

```
str1 = '人生苦短，我用Python。'
str1.encode('UTF-8')
#
b'\xe4\xba\xba\xe7\x94\x9f\xe8\x8b\xa6\xe7\x9f\xad\xef\xbc\x8c\xe6\x88\x91\xe7\x94\xa8Python
\xe3\x80\x82'
str1.encode('GB18030')
# b'\xc8\xcb\xc9\xfa\xbf\xe0\xb6\xcc\xa3\xac\xce\xd2\xd3\xc3Python\xa1\xa3'

str.maketrans("123","abc")    # {49: 97, 50: 98, 51: 99}

dict1 = {49: 97, 50: 98, 51: 99}
"123456".translate(dict1)     # "abc456"
```

例子：小写转大写（不使用内置函数）

不使用内置函数upper()，将输入的字符串中的小写字母转成大写字母，其它字符不变。

```
s = input()
t = ''
for i in range(len(s)):
    if 'a' <= s[i] <= 'z':
        t += chr(ord(s[i])-(ord('a')-ord('A')))
    else:
        t += s[i]
print(t)
```

例子：跳过含7的数字

打印1～100之间所有不含7的数字。

```
for i in range(1,101):
    if '7' not in str(i):
        print(i)
```

例子：去掉元音字母

输入一个字符串，去掉其中包含的元音字母后输出。

```
# No Vowels
# Demonstrates creating new strings with a for loop
message = input("Enter a message: ")
new_message = ""
VOWELS = "aeiou"
for letter in message:
    if letter.lower() not in VOWELS:
        new_message += letter
print("\nYour message without vowels is:", new_message)
```

字符串格式化

- % 操作符（最原始）
- 字符串的format()方法（Python2.6之后）
- f-String （Python3.6之后）
- 字符串模板 string template

% 操作符格式化字符串

- %格式化字符串是 python 最早的，也是能兼容所有版本的一种字符串格式化方法。格式为：

  `"xxxxxx %s xxxxxx %d " % (value1, value2)`

- 其中 `%s` 就是格式化符，意思是把后面的值格式化为字符类型，类似的格式化符还有`%d`（整数类型），`%f`（浮点数类型）等。
- 后面的`value1`,`value2`就是要格式化的值，不论是字符还是数值，都会被格式化为格式化符对应的类型。
- 如果只传一个参数，并且很确定参数类型不会触发异常，可以直接写这样：`"xxxxx %s xxxxx" % value`，不过不建议这样写，一是因为这样只能传递一个参数，二是如果value是元组或列表等类型，这样会触发TypeErrer 。建议提供一个单元素元组，就像`"xxxx %s " % (value,)`。

```
age = 10
name = 'Tom'
print("%s今年%d岁"%(name,age))
```

格式化符	意义
%d	返回要格式化对象的有符号十进制整数表示。
%i	返回要格式化对象的有符号十进制整数表示。
%o	返回要格式化对象的八进制表示。
%u	返回要格式化对象的无符号十进制整数表示。
%x	返回要格式化对象的十六进制表示。【如果要求的前导，使用'0x'】
%X	返回要格式化对象的十六进制表示。【如果要求的前导，使用'0X'】
%e	返回要格式化对象的浮点的科学计数的表示。【使用'e'】
%E	返回要格式化对象的浮点的科学计数的表示。【使用'E'】
%f	返回要格式化对象的浮点数表示。
%F	返回要格式化对象的浮点数表示。
%g	使用小写字母科学计数格式，如果指数小于-4或不少于精度,否则返回十进制格式。
%G	使用大写字母科学计数格式，如果指数小于-4或不少于精度,否则返回十进制格式。
%c	对0-255之间的整数返回其对应的ASCII码字符（其它整数报错），或者返回单个字符本身

格式化符	意义
%r	返回要格式化对象的**repr**()方法的返回值
%s	返回要格式化对象的**str**()方法的返回值
%%	即每两个%显示一个

- %s格式化符：将要格式化的值表中对应位置的元素，格式化为字符串，如果值元素不是字符串，将自动调用该元素的**str**()，以得到其字符串表示。
    - %ns决定对齐与填充：n为整数；当n>0时，左填充，当n<0时，右填充。
    - %.ns决定对字符串的截取：n为正整数

```
print("字符串：%s,%s,%s" % (1, 22.22, [1, 2]))
# 字符串：1,22.22,[1, 2]
print("字符串不足5位，左边补空格%5s。" % '2')
# 字符串不足5位，左边补空格    2。
print("字符串不足5位，右边补空格%-5s。" % '2')
# 字符串不足5位，右边补空格2    。
print("字符串宽10位，截取两位%10.2s。" % "hello.world")
# 字符串宽10位，截取两位         he。
```

- %d格式化符：将任何Python对象转化为整数，如果转化失败，则报错。
    - %nd：决定对齐与填充。n为整数；当n>0时，左填充，当n<0时，右填充。
    - %0nd：以数字0而不是默认的空格来作填充。
    - %+d：数字被转化后将保留正负号

```
print("整数：%d,%d,%d" % (1, 22.22, 33))
# 整数：1,22,33
print("整数不足5位，左边补空格：%5d。" % 22)
# 整数不足5位，左边补空格：    22。
print("整数不足5位，左边补0：%05d。" % 22)
# 整数不足5位，左边补0：00022。
print("整数不足5位，右边补空格%-5d。" % 22)
# 整数不足5位，右边补空格22    。
```

- %r格式化符：将任何Python对象转化成repr字符串，自动调用Python对象的**repr**()
    - %nr：决定对齐与填充。n为整数；当n>0时，左填充，当n<0时，右填充。
    - %.nr：决定截取字符数。n为正整数
- %o与%x格式化符：将十进制数值格式化为八进制与十六进制字符串；当有'#'时，格式化后的字符串带前导
    - 注意：如果要格式化的数值是浮点数，将先取整，再格式化

```
print("八进制 %o" % 222)
# 八进制 336
print("十六进制 %x" % 12)
# 十六进制 c
```

- %f格式化符：以浮点数格式化数值。
    - %nf：格式化数字后填充到固定宽度；当n>0时，左填充，当n<0时，右填充。【若不指定小数位数时，最小宽度8位，如'0.000000'】
    - %.nf：格式化数字后的小数位数；n>=0且为整数

```
print("浮点数：%f,%f" % (1, 22.22))
# 浮点数：1.000000,22.220000
print("浮点数保留两位小数：%.2f" % 22.222)
# 浮点数保留两位小数：22.22
print("浮点数保留两位小数，宽5位，不足补0：%05.2f" % 2.222)
# 浮点数保留两位小数，宽5位，不足补0：02.22
```

- %e格式化符：以浮点科学计数的方式转化数值，即10的指数幂形式
    - %ne：将格式化后的数字字符串进行填充到指定宽度；当n>0时，左填充，当n<0时，右填充。【如不指定小数位数，最小宽度是12位，如'0.000000e+00'】
    - %.ne：指定小数位数；n>=0且为整数。
- %c格式化符：将0-255之间的数字转化成对应的ASCII码字符，也可以转化单个字符，不过基本不做什么，直接返回

字符串格式化函数 format()

- 字符串类型格式化采用format()方法，基本使用格式是：

  <模板字符串>.format(<逗号分隔的参数>)

- format()方法中<模板字符串>除了包括参数序号，还可以包括格式控制信息。此时，<模板字符串>内部样式如下（注意：中间使用了一个冒号）：

  {<参数序号>: <格式控制标记>}

  其中，<格式控制标记>用来控制参数显示时的格式，包括：**<填充><对齐><宽度><,><.精度><类型>6 个字段**，这些字段都是可选的，可以组合使用，例如：

  ```
  s = "PYTHON"
  t = "Java"
  "{1:*^10} and {0:-^10}".format(s,t)
  # '***Java*** and --PYTHON--'
  ```

  格式控制字段逐一介绍如下。

:	<填充>	<对齐>	<宽度>	<,>	<.精度>	<类型>
	用于填充的单个字符	< 左对齐 > 右对齐 ^ 居中对齐	设定输出宽度	数字的千位分隔符， 适用于整数和浮点数	浮点数小数部分的精度 或字符串的最大输出长度	整数类型：b, c, d, o, x, X 浮点数类型：e, E, F, %

（1）<填充>

指<宽度>内除了参数外的字符采用什么方式表示，默认采用空格，可以通过<填充>更换。

（2）<对齐>

指参数在<宽度>内输出时的对齐方式，分别使用<、>和^三个符号表示左对齐、右对齐和居中对齐。

（3）<宽度>

指当前槽的设定输出字符宽度，如果该槽对应的format()参数长度比<宽度>设定值大，则使用参数实际长度。如果该值的实际位数小于指定宽度，则位数将被默认以空格字符补充。

```
s = "PYTHON"

"{0:30}".format(s)
# 'PYTHON                         '

"{0:>30}".format(s)
# '                         PYTHON'

"{0:*^30}".format(s)
# '************PYTHON************'

"{0:-^30}".format(s)
# '------------PYTHON------------'

"{0:3}".format(s)
#  'PYTHON'
```

（4）逗号（，）

<格式控制标记>中逗号（，）用于显示数字的千位分隔符，例如：

```
"{0:-^20,}".format(1234567890)
# '---1,234,567,890----'

"{0:-^20}".format(1234567890) #对比输出
# '-----1234567890-----'

"{0:-^20,}".format(12345.67890)
# '----12,345.6789-----'
```

（5）<.精度> # 注意：前面有一个小数点！

表示两个含义，由小数点（.）开头。对于浮点数，精度表示小数部分输出的有效位数。对于字符串，精度表示输出的最大长度。

```
"{0:.2f}".format(12345.67890)
# '12345.68'

"{0:H^20.3f}".format(12345.67890)
# 'HHHHH12345.679HHHHHH'

"{0:.4}".format("PYTHON")
# 'PYTH'
```

（6）<类型>

表示输出整数和浮点数类型的格式规则。

对于整数类型，输出格式包括6 种：

> b: 输出整数的二进制方式；
> c: 输出整数对应的 Unicode 字符；
> d: 输出整数的十进制方式；
> o: 输出整数的八进制方式；
> x: 输出整数的小写十六进制方式；
> X: 输出整数的大写十六进制方式；

对于浮点数类型，输出格式包括4 种：

> e: 输出浮点数对应的小写字母 e 的指数形式；
> E: 输出浮点数对应的大写字母 E 的指数形式；
> f: 输出浮点数的标准浮点形式；
> %: 输出浮点数的百分形式。

浮点数输出时尽量使用<.精度>表示小数部分的宽度，有助于更好控制输出格式。

```
"{0:e},{0:E},{0:f},{0:%}".format(3.14)
# '3.140000e+00,3.140000E+00,3.140000,314.000000%'

"{0:.2e},{0:.2E},{0:.2f},{0:.2%}".format(3.14)
# '3.14e+00,3.14E+00,3.14,314.00%'
```

f-string 格式化字符串

- f-string实际上是在运行时计算的表达式，而不是常量值。在Python源代码中，f-string是一个文字字符串，前缀为'f'，其中包含大括号内的表达式。表达式会将大括号中的内容替换为其值。例如：

```
import datetime
name = "zings"
age = 17
date = datetime.date(2019,7,18)
print(f'my name is {name}, this year is {date:%Y}. Next year, I\'m {age+1}')
# my name is zings, this year is 2019. Next year, I'm 18
```

- f-string采用 `{content:format}` 设置字符串格式，其中 `content` 是替换并填入字符串的内容，可以是变量、表达式或函数等，`format` 是格式描述符。采用默认格式时不必指定 `{:format}`，如上面例子所示只写 `{content}` 即可。
- 常用格式描述符的含义与作用，也是按照 **<填充><对齐><宽度><,><.精度><类型>** 这6个主要方面，但是还有所拓展。format一般的格式为:

  `fill]align][sign][#][0][width][grouping_option][.precision][type]`
  即
  `[填充字符][对齐方式][数字的正负号显示][#][0][宽度][千分位分组符号][.小数精度][类型]`

  (1) 填充 fill： 可以是任何的字符
  (2) 对齐 align： "<" | ">" | "=" | "^"
  (3) 符号 sign： 这个仅仅对数值有效 "+" | "-" | " "
  (4) 宽度 width： 是一个整数数值，表示多少宽度

（5）grouping_option：表示千分位的分隔符号，可以是 "_" | ","
（6）精度 precision： .数字
（7）类型 type： "b" | "c" | "d" | "e" | "E" | "f" | "F" | "g" | "G" | "n" | "o" | "s" | "x" | "X" | "%"

- 详细说明如下：

（2）对齐align

对其align	含义与作用
<	左对齐（字符串默认对齐方式）
>	右对齐（数值默认对齐方式）
^	居中

（3）符号sign

数字符号sign	含义与作用
+	负数前加负号（-），正数前加正号（+）
-	负数前加负号（-），正数前不加任何符号（默认）
（空格）	负数前加负号（-），正数前加一个空格

注：仅适用于数值类型。

（4）# 符号说明

# 描述符格式描述符	含义与作用
#	切换数字显示方式

注1：仅适用于数值类型。

注2：`#` 对不同数值类型的作用效果不同，详见下表：

数值类型	不加 `#`（默认）	加 `#`	区别
二进制整数	`'1111011'`	`'0b1111011'`	开头是否显示 `0b`
八进制整数	`'173'`	`'0o173'`	开头是否显示 `0o`
十进制整数	`'123'`	`'123'`	无区别
十六进制整数（小写字母）	`'7b'`	`'0x7b'`	开头是否显示 `0x`
十六进制整数（大写字母）	`'7B'`	`'0X7B'`	开头是否显示 `0`

（5）宽度与精度

宽度与精度相关格式描述符

格式描述符	含义与作用
`width`	整数 `width` 指定宽度
`0width`	整数 `width` 指定宽度，开头的 `0` 指定高位用 `0` 补足宽度
`width.precision`	整数 `width` 指定宽度，整数 `precision` 指定显示精度

注1：0width 不可用于复数类型和非数值类型，width.precision 不可用于整数类型。
注2：width.precision 用于不同格式类型的浮点数、复数时的含义也不同：用于 f、F、e、E 和 % 时 precision 指定的是小数点后的位数，用于 g 和 G 时 precision 指定的是有效数字位数（小数点前位数+小数点后位数）。
注3：width.precision 除浮点数、复数外还可用于字符串，此时 precision 含义是只使用字符串中前 precision 位字符。

（6）千位分隔符相关格式描述符

格式描述符	含义与作用
,	使用,作为千位分隔符
_	使用_作为千位分隔符

注1：若不指定 , 或 _，*则-string不使用任何千位分隔符，此为默认设置。*
注2：, 仅适用于浮点数、复数与十进制整数：对于浮点数和复数，, 只分隔小数点前的数位。
注3： 适用于浮点数、复数与二、八、十、十六进制整数：对于浮点数和复数，_ 只分隔小数点前的数位；对于二、八、十六进制整数，固定从低位到高位每隔四位插入一个 _（十进制整数是每隔三位插入一个 _）。

（7）类型描述

格式描述符	含义与作用	适用变量类型
`s`	普通字符串格式	字符串
`b`	二进制整数格式	整数
`c`	字符格式，按unicode编码将整数转换为对应字符	整数
`d`	十进制整数格式	整数
`o`	八进制整数格式	整数
`x`	十六进制整数格式（小写字母）	整数
`X`	十六进制整数格式（大写字母）	整数
`e`	科学计数格式，以 `e` 表示 `×10^`	浮点数、复数、整数（自动转换为浮点数）
`E`	与 `e` 等价，但以 `E` 表示 `×10^`	浮点数、复数、整数（自动转换为浮点数）

格式描述符	含义与作用	适用变量类型
`f`	定点数格式，默认精度（`precision`）是6	浮点数、复数、整数（自动转换为浮点数）
`F`	与 `f` 等价，但将 `nan` 和 `inf` 换成 `NAN` 和 `INF`	浮点数、复数、整数（自动转换为浮点数）
`g`	通用格式，小数用 `f`，大数用 `e`	浮点数、复数、整数（自动转换为浮点数）
`G`	与 `G` 等价，但小数用 `F`，大数用 `E`	浮点数、复数、整数（自动转换为浮点数）
`%`	百分比格式，数字自动乘上100后按 `f` 格式排版，并加 `%` 后缀	浮点数、整数（自动转换为浮点数）

常用的特殊格式类型：标准库 `datetime` 给定的用于排版时间信息的格式类型，适用于 `date`、`datetime` 和 `time` 对象。

格式描述符	含义	显示样例
`%a`	星期几（缩写）	`'Sun'`
`%A`	星期几（全名）	`'Sunday'`
`%w`	星期几（数字，`0` 是周日，`6` 是周六）	`'0'`
`%u`	星期几（数字，`1` 是周一，`7` 是周日）	`'7'`
`%d`	日（数字，以 `0` 补足两位）	`'07'`
`%b`	月（缩写）	`'Aug'`
`%B`	月（全名）	`'August'`
`%m`	月（数字，以 `0` 补足两位）	`'08'`
`%y`	年（后两位数字，以 `0` 补足两位）	`'14'`
`%Y`	年（完整数字，不补零）	`'2014'`
`%H`	小时（24小时制，以 `0` 补足两位）	`'23'`
`%I`	小时（12小时制，以 `0` 补足两位）	`'11'`
`%p`	上午/下午	`'PM'`
`%M`	分钟（以 `0` 补足两位）	`'23'`
`%S`	秒钟（以 `0` 补足两位）	`'56'`

格式描述符	含义	显示样例
`%f`	微秒（以 `0` 补足六位）	`'553777'`
`%z`	UTC偏移量（格式是 `±HHMM[SS]`，未指定时区则返回空字符串）	`'+1030'`
`%Z`	时区名（未指定时区则返回空字符串）	`'EST'`
`%j`	一年中的第几天（以 `0` 补足三位）	`'195'`
`%U`	一年中的第几周（以全年首个周日后的星期为第0周，以 `0` 补足两位）	`'27'`
`%w`	一年中的第几周（以全年首个周一后的星期为第0周，以 `0` 补足两位）	`'28'`
`%V`	一年中的第几周（以全年首个包含1月4日的星期为第1周，以 `0` 补足两位）	`'28'`

模板字符串 string template

```
from string import Template
number = 20090214
name = 'Fish'
templ_string = 'Hello $name, your number is $number.'
print(Template(templ_string).substitute(name=name, number=hex(number)))
# 'Hello Fish, your number is 0x1328d66.#
```

- 需要从Python的内置字符串模块中导入Template类
- 模板字符串不能使用格式说明符，需要手动转换
- 当用来处理程序用户生成的格式字符串时，这个简单的模板字符串机制比较安全

正则表达式与re模块

正则表达式是一个特殊的字符序列,它能检查一个字符串是否与某种模式匹配。学会使用Python自带的re模块编程非常有用，因为它可以帮我们快速检查一个用户输入的email或电话号码格式是否有效，也可以帮我们快速从文本中提取我们所需要的字符串。

正则表达式

正则表达式（Regular Expression，简称RE、regex）是强大的文本处理工具，用于匹配字符串模式。

正则表达式可以做什么?

- **查找**：在文本中找到特定模式的内容
- **替换**：将符合某种模式的文本替换为其他内容
- **验证**：检查输入的数据是否符合预期格式
- **提取**：从复杂文本中提取需要的信息

正则表达式的模式

正则表达式的模式可以包括以下内容：

- 字面值字符：例如字母、数字、空格等，可以直接匹配它们自身。
- 特殊字符：例如点号 `.`、星号 `*`、加号 `+`、问号 `?` 等，它们具有特殊的含义和功能。
- 字符类：用方括号 `[ ]` 包围的字符集合，用于匹配方括号内的任意一个字符。
- 元字符：例如 `\d`、`\w`、`\s` 等，用于匹配特定类型的字符，如数字、字母、空白字符等。
- 量词：例如 `{n}`、`{n,}`、`{n,m}` 等，用于指定匹配的次数或范围。
- 边界符号：例如 `^`、`$`、`\b`、`\B` 等，用于匹配字符串的开头、结尾或单词边界位置。

简单示例：

我们在写用户注册表单时，只允许用户名包含**字符、数字、下划线和连接字符 -**，并设置用户名的长度，我们就可以使用以下正则表达式来设定。

```
^[a-zA-Z0-9_-]{3,15}$
```

- `^` 表示匹配字符串的开头。
- `[a-zA-Z0-9_-]` 表示字符集，包含小写字母、大写字母、数字、下划线和连接字符 **-**。
- `{3,15}` 表示前面的字符集最少出现 3 次，最多出现 15 次，从而限制了用户名的长度在 3 到 15 个字符之间。
- `$` 表示匹配字符串的结尾。

正则表达式元字符和特性

字符匹配

- 普通字符：普通字符按照字面意义进行匹配，例如匹配字母 "a" 将匹配到文本中的 "a" 字符。
- 元字符：元字符具有特殊的含义，例如 `\d` 匹配任意数字字符，`\w` 匹配任意字母数字字符，`.` 匹配任意字符（除了换行符）等。

量词

- `*`：匹配前面的模式零次或多次。
- `+`：匹配前面的模式一次或多次。
- `?`：匹配前面的模式零次或一次。
- `{n}`：匹配前面的模式恰好 n 次。
- `{n,}`：匹配前面的模式至少 n 次。
- `{n,m}`：匹配前面的模式至少 n 次且不超过 m 次。

字符类

- `[ ]`：匹配括号内的任意一个字符。例如，`[abc]` 匹配字符 "a"、"b" 或 "c"。
- `[^ ]`：匹配除了括号内的字符以外的任意一个字符。例如，`[^abc]` 匹配除了字符 "a"、"b" 或 "c" 以外的任意字符。

边界匹配

- `^`：匹配字符串的开头。
- `$`：匹配字符串的结尾。
- `\b`：匹配单词边界。
- `\B`：匹配非单词边界。

分组和捕获

- `( )`：用于分组和捕获子表达式。
- `(?: )`：用于分组但不捕获子表达式。

特殊字符

- `\`：转义字符，用于匹配特殊字符本身。
- `.`：匹配任意字符（除了换行符）。
- `|`：用于指定多个模式的选择。

正则表达式示例

正则表达式是一种用于匹配和操作文本的强大工具。它由一系列字符和特殊字符组成的模式，用于描述要匹配的文本模式。以下是一些常见的正则表达式示例及其用途。

数字匹配

- **整数或小数**: *^[0-9]+.{0,1}[0-9]{0,2}$*
- **只能输入数字**: *^[0-9]*$*
- **只能输入n位的数字**: *^\d{n}$*
- **只能输入至少n位的数字**: *^\d{n,}$*
- **只能输入m~n位的数字**: *^\d{m,n}$*
- **只能输入有两位小数的正实数**: *^[0-9]+(.[0-9]{2})?$*
- **只能输入有1~3位小数的正实数**: *^[0-9]+(.[0-9]{1,3})?$*

字符匹配

- **只能输入由26个英文字母组成的字符串**: *^[A-Za-z]+$*
- **只能输入由26个大写英文字母组成的字符串**: *^[A-Z]+$*
- **只能输入由26个小写英文字母组成的字符串**: *^[a-z]+$*
- **只能输入由数字和26个英文字母组成的字符串**: *^[A-Za-z0-9]+$*
- **只能输入由数字、26个英文字母或者下划线组成的字符串**: *^\w+$*

特殊用途

- **验证用户密码**: *^[a-zA-Z]\w{5,17}$* 正确格式为：以字母开头，长度在6~18之间，只能包含字符、数字和下划线。
- **验证Email地址**: *^\w+([-+.]\w+)*@\w+([-.]\w+)*\.\w+([-.]\w+)*$*
- **验证电话号码**: *^(\(\d{3,4}-)|\d{3,4}-)?\d{7,8}$* 正确格式为：XXX-XXXXXXX、XXXX-XXXXXXXX、XXX-XXXXXXX、

XXX-XXXXXXXX、XXXXXXX和XXXXXXXX

- **验证身份证号（15位或18位数字）**：*^\d{15}|\d{18}$*
- **验证一年的12个月**：*^(0?[1-9]|1[0-2])$* 正确格式为：01～09和1～12
- **验证一个月的31天**：*^((0?[1-9])|((1|2)[0-9])|30|31)$* 正确格式为：01～09和1～31

其他示例

- **匹配中文字符**：*[\u4e00-\u9fa5]*
- **匹配双字节字符 (包括汉字在内)**：*[^\x00-\xff]*
- **匹配空行**：*\n[\s|]*\r*
- **匹配HTML标签**：*<(.*)>(.*)<\/\1>|<(.*)\/>*
- **匹配首尾空格**：*(^\s*)|(\s*$)*

这些正则表达式示例可以帮助你在不同的场景中快速匹配和操作文本，提高编码效率。

Python的re模块

Python通过re模块，提供了对正则表达式的支持。以下是一些常用的正则表达式操作及示例：

导入模块

```
import re
```

常用函数

Python自带的re模块主要包含如下6种方法

- re.compile: 编译一个正则表达式模式(pattern)
- re.match: 从头开始匹配, 使用group()方法可以获取第一个匹配值
- re.search: 用包含方式匹配，使用group()方法可以获取第一个匹配值
- re.findall: 用包含方式匹配，把所有匹配到的字符放到以列表中的元素返回多个匹配值
- re.sub: 匹配字符并替换
- re.split: 以匹配到的字符当做列表分隔符，返回列表

(1) re.match()

从字符串的开头开始匹配，若开头不符合，则返回None。

```
pattern = r"hello"
text = "hello world"
result = re.match(pattern, text)
if result:
    print("匹配成功:", result.group())  # 输出: 匹配成功: hello
else:
    print("匹配失败")
```

(2) re.search()

在整个字符串中搜索第一个匹配项。

```
pattern = r"world"
text = "hello world"
result = re.search(pattern, text)
if result:
    print("匹配成功:", result.group())  # 输出: 匹配成功: world
else:
    print("匹配失败")
```

(3) re.findall()

返回所有匹配的子串列表。

```
pattern = r"\d+"  # 匹配数字
text = "价格是100元，折扣20元"
result = re.findall(pattern, text)
print(result)  # 输出: ['100', '20']
```

(4) re.sub()

替换匹配的内容。

```
pattern = r"\d+"
text = "价格是100元，折扣20元"
result = re.sub(pattern, "X", text)
print(result)  # 输出: 价格是X元，折扣X元
```

(5) re.split()

根据匹配的模式分割字符串。

```
pattern = r"\s+"  # 匹配空白字符
text = "hello   world python"
result = re.split(pattern, text)
print(result)  # 输出: ['hello', 'world', 'python']
```

(6) re.compile()

为了提高效率，可以使用re.compile()预编译正则表达式。

```
pattern = re.compile(r"\d+")
text = "价格是100元，折扣20元"
result = pattern.findall(text)
print(result)  # 输出: ['100', '20']
```

案例：验证邮箱地址

```
email_pattern = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
email = "example@test.com"
if re.match(email_pattern, email):
    print("邮箱格式正确")
else:
    print("邮箱格式错误")
```